Effat University — College of Engineering
Department of Computer Science
CS3081 Artificial Intelligence

LAB 3: Propositional Logic & Model Checking

Group Challenge

Instructor	Dr. Naila Marir	**Duration**	1 hour 30 minutes
Semester	Fall 2026	**Group size**	3–4 students
Submission	One GitHub link per group	**Total marks**	100

Group No.			
Member name	**Student ID**	**Member name**	**Student ID**
Ali Ahmed	**S24109004**		

4. Task 1 — Understand the Code (30 marks, 20 min)

1. `harry.py`: Truth table for `rain`, `hagrid`, `dumbledore` (8 rows)

Knowledge Base: `(¬rain ⇒ hagrid) ∧ (hagrid ∨ dumbledore) ∧ ¬(hagrid ∧ dumbledore) ∧ dumbledore`

Row	rain	hagrid	dumbledore	¬rain ⇒ hagrid	hagrid ∨ dumbledore	¬(hagrid ∧ dumbledore)	KB	Evaluation Notes
1	F	F	F	F	F	T	F	`¬rain ⇒ hagrid`, `hagrid ∨ dumbledore`, and `dumbledore` are False
2	F	F	T	F	T	T	F	`¬rain ⇒ hagrid` is False (since `¬rain` is True and `hagrid` is False)
3	F	T	F	T	T	T	F	Fact `dumbledore` is False
4	F	T	T	T	T	F	F	`¬(hagrid ∧ dumbledore)` is False (both cannot be True)
5	T	F	F	T	F	T	F	`hagrid ∨ dumbledore` and `dumbledore` are False
6 ✓	**T**	**F**	**T**	**T**	**T**	**T**	**T**	**Knowledge base is True (all four sentences hold)**
7	T	T	F	T	T	T	F	Fact `dumbledore` is False
8	T	T	T	T	T	F	F	`¬(hagrid ∧ dumbledore)` is False (both cannot be True)

Explanation of why the program prints `True`: The function `model_check(knowledge, query)` checks all 8 rows of the truth table to verify whether `query` (`rain`) is True in every row where `knowledge` is True. As marked in the table above, **Row 6** (`rain = T, hagrid = F, dumbledore = T`) is the only row where the knowledge base is True, and in that row `rain` is True. Therefore, the knowledge base entails `rain`, and the program prints `True`.

Output (`harry.py`)

```
PS D:\cs 3081\labs\lab2> python harry.py
True
```

2. `clue.py`: Explain in plain English why `MsScarlet`, `library` and `knife` must be true

- **`MsScarlet`:** One of the three suspects (`ColMustard`, `ProfPlum`, or `MsScarlet`) must be in the envelope. Our initial cards rule out `ColMustard`, and a revealed card rules out `ProfPlum`, so the suspect must be `MsScarlet`.
- **`library`:** One of the three rooms (`ballroom`, `kitchen`, or `library`) must be in the envelope. Our initial cards rule out the `kitchen`, and a revealed card rules out the `ballroom`, so the room must be the `library`.
- **`knife`:** One of the three weapons (`knife`, `revolver`, or `wrench`) must be in the envelope. The unknown card clue (`¬MsScarlet ∨ ¬library ∨ ¬wrench`) means at least one of those three is false. Since `MsScarlet` and `library` are already proven true, `wrench` must be false. Because `revolver` was ruled out by our initial hand, the weapon must be the `knife`.

Output (`clue.py`)

```
PS D:\cs 3081\labs\lab3> python clue.py
MsScarlet: YES
library: YES
knife: YES
```

5. Task 2 — Change the Knowledge (30 marks, 20 min)

1. `harry.py`: Delete the last fact `dumbledore` from the knowledge base

- **(a) Prediction:** `False`.
- **(b) Result:** `False`.
- **(c) Explanation:** When `dumbledore` is removed from the knowledge base, Row 3 (`rain = F, hagrid = T, dumbledore = F`) also satisfies all remaining rules in the knowledge base, even though `rain` is False in that row. Because `rain` is no longer True in every model where the knowledge base is True, the knowledge base does not entail `rain`, and the program prints `False`.

Code (`harry.py`)

```
from logic import *

rain = Symbol("rain")
hagrid = Symbol("hagrid")
dumbledore = Symbol("dumbledore")

knowledge = And(
    Implication(Not(rain), hagrid),
    Or(hagrid, dumbledore),
    Not(And(hagrid, dumbledore))
)

print(model_check(knowledge, rain))
```

Output (`harry.py`)

```
PS D:\cs 3081\labs\lab3> python harry.py
False
```

2. `puzzle.py`: Delete the clue `MinervaGryffindor`, then add one new clue of your own

- **(a) Prediction:** No person can be placed in a house (nothing will be printed).
- **(b) Result:** Nothing is printed.
- **(c) Explanation:** Without `MinervaGryffindor`, Gilderoy can be in either Gryffindor or Ravenclaw, Pomona can be in any house except Slytherin, and Minerva and Horace can take the remaining houses. Because multiple valid house arrangements satisfy the knowledge base, no single `PersonHouse` symbol holds in all models.

Output (After Deleting `MinervaGryffindor`)

```
PS D:\cs 3081\labs\lab3> python puzzle.py
(no output)
```

New Clue Added (`Symbol("HoraceRavenclaw")`): Adding the clue that Horace is in Ravenclaw makes the solution unique again: since Horace takes Ravenclaw and Gilderoy must be in Gryffindor or Ravenclaw, Gilderoy must be in **Gryffindor**. Only Hufflepuff and Slytherin remain for Pomona and Minerva. Since Pomona cannot be in Slytherin, Pomona is placed in **Hufflepuff**, leaving Minerva in **Slytherin**.

Code (`puzzle.py` Clue Change)

```
# Task 2.2: Deleted clue MinervaGryffindor
# knowledge.add(
#     Symbol("MinervaGryffindor")
# )

# Task 2.2: New clue that restores a unique house assignment
knowledge.add(
    Symbol("HoraceRavenclaw")
)
```

Output (With New Clue `HoraceRavenclaw`)

```
PS D:\cs 3081\labs\lab3> python puzzle.py
GilderoyGryffindor
PomonaHufflepuff
MinervaSlytherin
HoraceRavenclaw
```

6. Task 3 — Group Challenge: Knights & Knaves (30 marks, 30 min)

Code (`knights.py` — Puzzles 0, 1, 2, and Bonus Puzzle 3)

```
from logic import *

AKnight = Symbol("A is a Knight")
AKnave = Symbol("A is a Knave")

BKnight = Symbol("B is a Knight")
BKnave = Symbol("B is a Knave")

CKnight = Symbol("C is a Knight")
CKnave = Symbol("C is a Knave")

# Puzzle 0
# A says "I am both a knight and a knave."
knowledge0 = And(
    Or(AKnight, AKnave),
    Not(And(AKnight, AKnave)),
    Implication(AKnight, And(AKnight, AKnave)),
    Implication(AKnave, Not(And(AKnight, AKnave)))
)

# Puzzle 1
# A says "We are both knaves."
# B says nothing.
knowledge1 = And(
    Or(AKnight, AKnave),
    Not(And(AKnight, AKnave)),
    Or(BKnight, BKnave),
    Not(And(BKnight, BKnave)),
    Implication(AKnight, And(AKnave, BKnave)),
    Implication(AKnave, Not(And(AKnave, BKnave)))
)

# Puzzle 2
# A says "We are the same kind."
# B says "We are of different kinds."
same_kind = Or(And(AKnight, BKnight), And(AKnave, BKnave))
diff_kind = Or(And(AKnight, BKnave), And(AKnave, BKnight))
```

```
knowledge2 = And(
    Or(AKnight, AKnave),
    Not(And(AKnight, AKnave)),
    Or(BKnight, BKnave),
    Not(And(BKnight, BKnave)),
    Implication(AKnight, same_kind),
    Implication(AKnave, Not(same_kind)),
    Implication(BKnight, diff_kind),
    Implication(BKnave, Not(diff_kind))
)

# Puzzle 3 (Bonus)
# A says either "I am a knight." or "I am a knave.", but you don't know which.
# B says "A said 'I am a knave'."
# B says "C is a knave."
# C says "A is a knight."
a_said_knight = And(
    Implication(AKnight, AKnight),
    Implication(AKnave, Not(AKnight))
)
a_said_knave = And(
    Implication(AKnight, AKnave),
    Implication(AKnave, Not(AKnave))
)

knowledge3 = And(
    Or(AKnight, AKnave),
    Not(And(AKnight, AKnave)),
    Or(BKnight, BKnave),
    Not(And(BKnight, BKnave)),
    Or(CKnight, CKnave),
    Not(And(CKnight, CKnave)),
    Or(a_said_knight, a_said_knave),
    Implication(BKnight, a_said_knave),
    Implication(BKnave, Not(a_said_knave)),
    Implication(BKnight, CKnave),
    Implication(BKnave, Not(CKnave)),
    Implication(CKnight, AKnight),
    Implication(CKnave, Not(AKnight))
)

def main():
    symbols = [AKnight, AKnave, BKnight, BKnave, CKnight, CKnave]
    puzzles = [
        ("Puzzle 0", knowledge0),
        ("Puzzle 1", knowledge1),
        ("Puzzle 2", knowledge2),
        ("Puzzle 3", knowledge3)
    ]
    for puzzle, knowledge in puzzles:
        print(puzzle)
        if len(knowledge.conjuncts) == 0:
            print("    Not yet implemented.")
        else:
            for symbol in symbols:
                if model_check(knowledge, symbol):
                    print(f"    {symbol}")

if __name__ == "__main__":
    main()
```

Output (`knights.py`)

```
PS D:\cs 3081\labs\lab3> python knights.py
Puzzle 0
    A is a Knave
Puzzle 1
    A is a Knave
    B is a Knight
Puzzle 2
    A is a Knave
    B is a Knight
Puzzle 3
    A is a Knight
    B is a Knave
    C is a Knight
```

Explanations

- **1. Puzzle 0 (`A is a Knave`):** A says "I am both a knight and a knave." Because no person can be both at the same time, the statement is always false. Since knights always tell the truth, A cannot be a knight, so **A is a Knave**.
- **2. Puzzle 1 (`A is a Knave`, `B is a Knight`):** A says "We are both knaves." If A were a knight, the statement would have to be true, meaning A is a knave—a contradiction. Thus **A is a Knave**. Since A's statement is false and A is a knave, B cannot be a knave, so **B is a Knight**.
- **3. Puzzle 2 (`A is a Knave`, `B is a Knight`):** A says "We are the same kind" and B says "We are of different kinds." If A were a knight, they would be the same kind, making B a knight too—but then B's statement ("different kinds") would be false. Therefore **A is a Knave**, which means they are of different kinds and **B is a Knight**.

Bonus (+10 marks) — Puzzle 3 (`A is a Knight`, `B is a Knave`, `C is a Knight`): No islander can ever say "I am a knave" (a knight would never lie and say it, and a knave would never tell the truth and admit it). Therefore, B's claim that A said "I am a knave" must be false, proving **B is a Knave**. Because B always lies, B's second claim ("C is a knave") is also false, which means **C is a Knight**. Finally, since C tells the truth, C's statement ("A is a knight") is true, so **A is a Knight**.